# Ders 13 Genişletilmiş Notu: Turing Makinelerini Birleştirme, Turing Tezi ve Turing Makinesi Çeşitleri

## Ders 13 Genişletilmiş Notu: Turing Makinelerini Birleştirme, Turing Tezi ve Turing Makinesi Çeşitleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Fonksiyon Hesaplayan Turing Makinelerine Dönüş
- 2. Karşılaştırma Fonksiyonu
- 3. Turing Makinelerini Birleştirme ve Blok Diyagramları
- 4. Turing Tezi
- 5. Algoritmanın Turing Makinesiyle Tanımı
- 6. Standart Turing Makinesi Modeli
- 7. Turing Makinesi Çeşitlerine Genel Bakış
- 8. Güç Kavramı
- 9. Aynı Güç ve Benzetim Yöntemi
- 10. Kalma Seçenekli Turing Makinesi
- 11. Yarı Sonsuz Şeritli Turing Makinesi
- 12. Çok Şeritli Turing Makinesi
- 13. Çok Boyutlu Turing Makinesi
- 14. Deterministik Olmayan Turing Makinesi
- 15. Karmaşıklık Sınıfları: P ve NP


### 1. Fonksiyon Hesaplayan Turing Makinelerine Dönüş

Geçen hafta Turing makineleri (Turing machine) işlenmiş ve bazı fonksiyonları hesaplayan makine örnekleri görülmüştü. Bu örneklerden ikisi $f(x, y) = x + y$ fonksiyonunu ve $f(x) = 2x$ fonksiyonunu hesaplayan makinelerdi. $2x$ için hem makinenin tamamı görülmüş hem de bir sözde kod (pseudocode) verilmişti.

Hoca bu dersten itibaren yöntemin değişeceğini söyler. Turing makineleri giderek karmaşıklaşacağı için artık durum makineleri (state machine) yazılmayacak, yalnız sözde kod verilecektir.

Fonksiyonların bir özelliği de birleştirilebilmeleridir. Bir fonksiyonu hesaplarken başka bir fonksiyon kullanılabilir. Hoca bu konuya geçmeden önce bir örnek daha verir.

### 2. Karşılaştırma Fonksiyonu

#### 2.1 Fonksiyon ve Gösterim

Örnek, parçalı (piecewise) bir fonksiyondur:

$$f(x, y) = \begin{cases} 1 & x > y \\ 0 & x \leq y \end{cases}$$

Hoca bu fonksiyonun içindeki koşullara dikkat çeker: burada "eğer"ler vardır. Örneğin iddiası, bu $f(x, y)$ fonksiyonunun hesaplanabilir (computable) olduğudur.

Burada tam sayılarla çalışılır ve tam sayılar yine tekli gösterimle (unary representation) yazılır. Örneğin `111`, 3 demektir. Turing makinesinde her şey bir dizgidir ve elimizde büyük bir şerit vardır. Bu yüzden önce giriş ve çıkış dizgilerinin biçimi kararlaştırılmalıdır:

- **Giriş:** $x$ ve $y$ yalnız 1'lerden oluştuğu için aralarına ayırıcı olarak 0 konur. Giriş $1^x\,0\,1^y$ biçimindedir.
- **Çıkış:** Dizgi olarak tek bir `0` ya da `1`.

### 2.2 Sözde Kod

Şeritte bir miktar 1, ayırıcı 0 ve yine bir miktar 1 vardır. 0'ın solu $x$ tarafı, sağı $y$ tarafıdır. Yapılacak iş, $x$ tarafındaki birinci 1'i $y$ tarafındaki bir 1 ile eşleştirmek, sonra ikinci 1'i eşleştirmek ve bunu $x$'in ya da $y$'nin bütün 1'leri eşleşene kadar sürdürmektir. Bu bir döngüdür. Döngüden çıkıldığında $x$ tarafında eşleşmemiş bir 1 kaldıysa $x$'te daha çok 1 var demektir, yani $x$ büyüktür ve parçalı fonksiyonun ilk durumu geçerlidir. Sonucu döndürmek için şeritteki her şey silinir ve 1 yazılır. Aksi durumda yine her şey silinir, ama bu kez 0 yazılır.

```
tekrarla
    x'ten bir 1'i y'den bir 1 ile eşleştir
ta ki x'in ya da y'nin bütün 1'leri eşleşene kadar

eğer x'ten eşleşmemiş bir 1 kaldıysa
    şeridi sil, 1 yaz
değilse
    şeridi sil, 0 yaz
```

Hoca bu sözde kodu "Turing makinesi" olarak sunar. Durum diyagramını vermez, bunun bir Turing makinesiyle yapılabileceğini gözümüzde canlandırmamızı ister.

### 2.3 Eşleştirme Daha Önce Görüldü

Eşleştirme adımı yeni bir şey değildir. Hoca önce bunun toplama makinesinde yapıldığını söyler, sonra örneklere bakıp asıl benzerliğin $a$ ile $b$'leri eşleştiren makinede olduğunu belirtir. O makinede $a$ sayısı ile $b$ sayısının aynı olması gerekiyordu, yani Turing makineleriyle iki tarafın sayısı eşleştirilebiliyordu. Burada da aynı tür makine, iki taraftaki 1'leri eşleştirmek için kullanılır.

## 3. Turing Makinelerini Birleştirme ve Blok Diyagramları

Elimizde bu tür fonksiyonlar olunca onları birleştirebiliriz (combining Turing machines). Bunun için blok diyagramları (block diagram) kullanılır. Bu derste her blok bir Turing makinesidir. Bir Turing makinesi giriş alır, çıkış döndürür; yani tıpkı bir fonksiyon gibidir. Böylece sözde kod yazmanın yanında blok diyagramı çizmek de mümkün olur.

### 3.1 Örnek

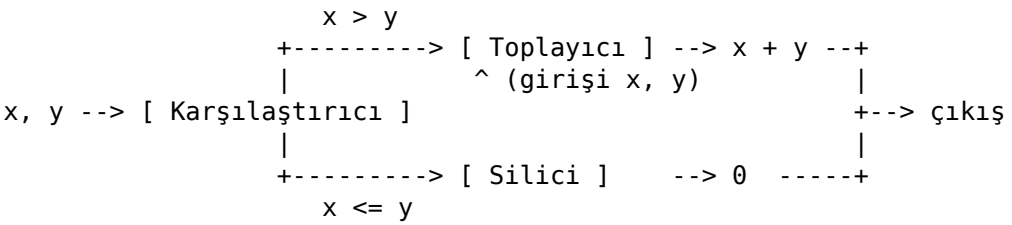
$$f(x, y) = \begin{cases} x + y & x > y \\ 0 & x \leq y \end{cases}$$

Bu fonksiyon, Turing makinelerini birleştirerek elde edilen yeni bir Turing makinesiyle hesaplanır, dolayısıyla hesaplanabilirdir. Parçalarının ikisi zaten elimizdedir: geçen hafta görülen toplama fonksiyonu ve az önce görülen karşılaştırma fonksiyonu. İkisi de hesaplanabilir, ikisi için de Turing makinesi vardır.

Diyagram şöyle kurulur:

- Önce bir karşılaştırıcı (comparator) gerekir. Bu bir Turing makinesidir, isterseniz fonksiyon da diyebilirsiniz. Girişi $x$ ve $y$'dir.
- $x > y$ ise toplayıcı (adder) çalışır. Toplayıcının girişi de $x$ ve $y$'dir, çıkışı $x + y$'dir.

- $x \leq y$ ise silici (eraser) çalışır. Silici bir dizgi alır, onu siler ve yerine 0 yazar. Yani yalnız silmekle kalmaz. Bu adlar bizim verdiğimiz kısa adlardır.
- Toplayıcı ya da siliciden hangisi çalıştıysa onun çıkışı, bütün makinenin çıkışıdır.




Ortaya çıkan şey bir akış diyagramına benzer. Turing makineleri bu fonksiyonlar aracılığıyla birleştirilebilir ve bu birleşim böyle diyagramlarla gösterilebilir.

> **Ders sonrası not:** 2. bölümdeki karşılaştırma makinesi şeridi silip yalnız 0 ya da 1 bıraktığı için, bu haliyle çalışırsa toplayıcıya $x$ ve $y$ kalmaz. Diyagramda toplayıcıya $x, y$'nin ayrıca gelmesi bu yüzden önemlidir. Tek şeritli bir makinede bu, karşılaştırıcının girişi koruması ya da işe başlamadan girişin bir kopyasını alması demektir.

## 4. Turing Tezi

Hoca sıradaki konunun önemli olduğunu söyler: Turing tezi (Turing's thesis), diğer adıyla Church-Turing tezi. Tez şunu söyler:

> Mekanik yollarla (mechanical means) yapılan her hesaplama bir Turing makinesiyle yapılabilir.

Bunun anlamı, bilgisayarlarda yapılan her hesaplamanın bir Turing makinesiyle de yapılabilmesidir.

> **Ek bilgi (derste anlatılmadı):** Hoca bu önermeyi teorem olarak değil tez olarak adlandırdı ve ispat vermedi. "Mekanik yol" ifadesi biçimsel olarak tanımlanmadığı sürece tez, matematiksel bir teorem gibi ispatlanamaz. Kesin olan kısım, tanımlı bir model için yapılan benzetimlerdir. Bu dersin ikinci yarısındaki ispatlar böyledir.

## 5. Algoritmanın Turing Makinesiyle Tanımı

Turing tezinden algoritmanın (algorithm) bir tanımı da çıkar:

> Bir problem için algoritma, o problemi çözen bir Turing makinesidir.

Gerekçe şudur. Algoritma, bir problemi çözmek için izlenecek mekanik yolun adımlarını anlatır. Sözde kod gibidir, adımları vardır. Bu adımlar da kolayca Turing makinesine çevrilebilir. O halde bir problemi çözen algoritmanın olması için o problemi çözen bir Turing makinesinin olması gerekir.

Bu sayede bir algoritmanın var olup olmadığından, Turing makinesinin var olup olmadığı üzerinden söz edilebilir. "Bir algoritma vardır" demek, "o algoritmayı yürüten bir Turing makinesi vardır" demektir. Yani o adımları yürütecek bir Turing makinesi yazılabilir.

Hoca örnek olarak daha önce verilen Turing makinesi sözde kodunu gösterir. Bu aslında bir algoritmadır, çünkü fonksiyonu hesaplamak için izlenen adımlardan oluşur. Bu fonksiyonu hesaplayan bir algoritma vardır, çünkü gerekli adımları yürüten bir Turing makinesi tarif edilebilir.

> **Hocanın vurgusu:** Bu kısım önemlidir. Size "algoritma nedir?" diye sorulursa elbette "problemi çözen adımlar" diyeceksiniz. Ama "algoritma, bir Turing makinesiyle tarif edilir" de diyebilirsiniz. Bir problem için algoritmanın var olup olmadığından, o problem için Turing makinesinin var olup olmadığı üzerinden konuşabilirsiniz.
>
> **Ek bilgi (derste anlatılmadı):** Yaygın tanımda bir algoritmanın her girişte durması beklenir. Bu yüzden bir algoritmaya karşılık gelen Turing makinesi her girişte duran bir makinedir. Bazı girişlerde sonsuz döngüye girebilen bir tanıyıcı (recognizer) bu tanımı karşılamaz.

## 6. Standart Turing Makinesi Modeli

Çeşitlere geçmeden önce hoca şimdiye kadar görülen modeli standart model (standard model) diye adlandırır ve özelliklerini hatırlatır:

- **Şerit:** Şerit sonsuzdur ve iki tarafa doğru da sonsuzdur. Girişin hemen önünde ve hemen arkasında boş semboller (blank) vardır.
- **Kafa:** Bir okuma-yazma kafası (read-write head) vardır. Hesaplamanın başında girişin ilk sembolünü gösterir.
- **Hareket:** Kafa her hesaplama adımında ya sola ya sağa gider.
- **Kontrol birimi:** Bütün durumları tutan bir kontrol birimi (control unit) vardır ve bu birim deterministiktir (deterministic). Yani standart Turing makinesi deterministiktir.

Bundan sonra "standart Turing makinesi" ya da yalnızca "Turing makinesi" denince bu model anlaşılacaktır.

## 7. Turing Makinesi Çeşitlerine Genel Bakış

Hoca standart modelin şu çeşitlerini (variations) sayar:

- **Kalma seçenekli (stay-option) Turing makinesi:** Okuma-yazma kafası aynı yerde kalabilir. Sola ya da sağa gitmek zorunda değildir.
- **Yarı sonsuz (semi-infinite) şeritli Turing makinesi:** Şerit yalnız bir tarafa, örneğin yalnız sağa doğru sonsuzdur. Solda sonsuz değildir.
- **Çok şeritli (multi-tape) Turing makinesi:** Birden çok şeridi vardır.
- **Çok boyutlu (multi-dimensional) Turing makinesi:** Şerit iki ya da daha çok boyutludur. Hocanın benzetmesiyle basit bir şerit değil, bir sayfa gibidir.
- **Deterministik olmayan (nondeterministic) Turing makinesi.**

Bu çeşitlerin her biri yeni bir Turing makinesi sınıfı (class) tanımlar. Bütün kalma seçenekli Turing makineleri, kalma seçenekli Turing makinesi sınıfını oluşturur. Bütün yarı sonsuz şeritli makineler de yarı sonsuz şeritli makine sınıfını oluşturur. Dersin ana sonucu, bu makinelerin hepsinin aynı güçte olduğudur.

## 8. Güç Kavramı

Burada güç (power), dil kabul edebilme yeteneğidir. Bu derste bir makine sınıfının gücü, o sınıfın tanıyabildiği dil sınıfını gösterir.

Hocanın ilk örneği aşağı itmeli otomatlardır (pushdown automata). Aşağı itmeli otomatlar, sonlu otomatlardan (finite automata) daha güçlüdür. Çünkü sonlu otomatlar düzenli dilleri (regular languages), aşağı itmeli otomatlar bağlamdan bağımsız dilleri (context-free languages) kabul eder. Düzenli diller, bağlamdan bağımsız dillerin alt kümesidir. Yani aşağı itmeli otomatlar, sonlu otomatların kabul edebildiği bütün dilleri ve başka bazı dilleri de kabul edebilir.

> **Ders sonrası not:** Hoca önce "bağlamdan bağımsız diller düzenli dillerden daha güçlüdür" diye yazdı, sonra kendisi düzeltti. Güç burada makine sınıfına aittir: doğrusu "aşağı itmeli otomatlar sonlu otomatlardan güçlüdür".

Sıradaki adımda standart Turing makineleri aşağı itmeli otomatlardan daha güçlüdür. Turing makineleri, Turing ile tanınabilir dilleri (Turing-recognizable languages) tanır. Bu diller bağlamdan bağımsız dillerin üst kümesidir, yani her bağlamdan bağımsız dil Turing ile tanınabilirdir.

> **Ders sonrası not:** Hoca bu cümleye "Turing makineleri standart Turing makinelerinden daha güçlüdür" diye başladı. Doğrusu, hemen ardından tamamladığı gibi, "standart Turing makineleri aşağı itmeli otomatlardan daha güçlüdür".
>
> **Ek bilgi (derste anlatılmadı):** "Daha güçlü" demek için üst kümenin kesin olması gerekir. $\{a^n b^n c^n \mid n \geq 0\}$ dili bağlamdan bağımsız değildir ama bir Turing makinesiyle tanınır. Bu yüzden kapsama kesindir.

Şimdi soru şudur: Kalma seçenekli, yarı sonsuz şeritli, çok şeritli, çok boyutlu ya da deterministik olmayan Turing makinelerinden herhangi biri standart Turing makinesinden daha

güçlü müdür? Cevap hayırdır. Bu çeşitlerin hepsi standart Turing makinesiyle aynı güçtedir. Örneğin deterministik olmayan Turing makinesinin gücü, deterministik olan standart Turing makinesinin gücüyle aynıdır. Bunu ispatlamak için benzetim (simulation) denen yöntem gerekir.

# 9. Aynı Güç ve Benzetim Yöntemi

## 9.1 Aynı Güç Ne Demektir?

İki sınıfın aynı güçte olması, iki sınıfın da aynı diller kümesini kabul etmesi demektir. Daha matematiksel yazarsak:

> Birinci sınıftaki her $M_1$ makinesi için ikinci sınıfta $L(M_1) = L(M_2)$ olan bir $M_2$ makinesi vardır. Bunun tersi de doğrudur.

Her Turing makinesi çeşidi için, o çeşidin sınıfının standart Turing makinesiyle aynı güçte olduğu bu özellik üzerinden ispatlanacaktır.

## 9.2 Benzetim Yöntemi

Benzetim, makinelerin aynı güçte olduğunu ispatlamak için kullanılan bir tekniktir. Elimizde birinci sınıftan özgün bir $M_1$ makinesi olsun. Birinci sınıf, örneğin deterministik ya da deterministik olmayan Turing makineleri olabilir. İkinci sınıfta öyle bir $M_2$ yazarsak ki $M_1$ ne yapıyorsa $M_2$ de bir şekilde onu yapsın, $M_2$'ye benzetim makinesi (simulation machine) denir. Böyle bir makinenin her zaman yazılabildiğini gösterirsek, ikinci sınıfın makineleri birinci sınıfın makinelerini benzetebiliyor demektir.

> **Ders sonrası not:** Hoca bu noktada "demek ki ikinci sınıf birinci sınıfla aynı güçtedir" dedi. Tek yönlü benzetim yalnız ikinci sınıfın en az birinci sınıf kadar güçlü olduğunu gösterir. Aynı güç için iki yönlü benzetim gerekir. Hocanın sonraki ispatları da bu yüzden iki adımlıdır.

Benzetim sırasında özgün makinedeki konfigürasyonlar (configuration), benzetim makinesindeki konfigürasyonlara karşılık gelir. $M_1$'deki bir hesaplama, anlık betimlemeler (instantaneous description) dizisidir:

$$C_1 \vdash C_2 \vdash C_3 \vdash \cdots \vdash C_n$$

Son konfigürasyon kabul eden bir konfigürasyondur. $M_2$'de bunların her birine karşılık gelen bir konfigürasyon vardır:

$$C_1^*,\ C_2^*,\ C_3^*,\ \ldots,\ C_n^*$$

Yani aynı hesaplama $M_2$'de de yapılır. Son kabul konfigürasyonu da iki tarafta aynıdır, ya da birbirine karşılık gelir. Her hesaplama bir girişle başlar. Aynı işler yapıldığında aynı dizgiler kabul edilir. Dolayısıyla $L(M_1) = L(M_2)$ olur. Hoca bunun bir örnekle yapıldığında aslında çok kolay görüleceğini söyler.

# 10. Kalma Seçenekli Turing Makinesi

## 10.1 Tanım

Bu makinede her şey standart Turing makinesiyle aynıdır, tek fark okuma-yazma kafasıdır. Kafa üç şey yapabilir:

- Sola gider (L).
- Sağa gider (R).
- Aynı yerde kalır (S).

Şerit yine iki tarafa da sonsuzdur. Hoca bu makine için bu bölümde STOTM, standart Turing makinesi için de STM kısaltmasını kullanır.

### 10.2 Teorem ve İspatın Yapısı

**Teorem:** Kalma seçenekli Turing makineleri, standart Turing makineleriyle aynı güçtedir.

Bu bir eşdeğerlik olduğu için iki şey ispatlanmalıdır:

- **1. adım:** Kalma seçenekli Turing makineleri standart Turing makinelerini benzetir.
- **2. adım:** Standart Turing makineleri kalma seçenekli Turing makinelerini benzetir.

### 10.3 Birinci Adım

Bu adım için hiçbir şey yapmak gerekmez, çünkü önemsiz (trivial) bir durumdur. Her standart Turing makinesi aynı zamanda kalma seçenekli bir Turing makinesidir. Standart makinenin kafası sağa gidebilir, kalma seçenekli makinenin kafası da gidebilir. Standart makinenin kafası sola gidebilir, kalma seçenekli makinenin kafası da gidebilir. İki hareket de kalma seçenekli makinenin tanımında zaten vardır. Bu yüzden kalma seçenekli makine, bir standart makineyi benzetebilir.

### 10.4 İkinci Adım

Odaklanılması gereken yer farktır, yani kafanın yerinde kalma seçeneğidir. Kalma seçenekli makinede şöyle bir geçiş olsun: $q_1$'den $q_2$'ye geçilirken $a$ okunur, $b$ yazılır ve kafa yerinde kalır.

$$\delta(q_1, a) = (q_2, b, S)$$

Standart makinede kafa yerinde kalamaz. Bunun yerine $q_2$ olmayan yeni bir ara duruma geçilir ve o durumdan $q_2$'ye bir geçiş eklenir. Yerinde kalma, önce sola gidip sonra sağa geri dönerek benzetilir.

Hocanın örneği şöyledir. Şeritte `bab` vardır, kafa ortadaki $a$'dadır ve makine $q_1$'dedir. Kalma seçenekli makine bir adımda `bbb` şeridine, kafa yine ortada ve durum $q_2$ olacak şekilde geçer. Standart makinenin de aynı yerden başlayıp aynı yere varması gerekir, ama arada ek adımlar yapılabilir:

| Adım | Şerit | Kafanın gösterdiği hücre | Durum | Yapılan |
|---|---|---|---|---|
| Başlangıç | `bab` | ortadaki `a` | $q_1$ | |
| 1 | `bbb` | soldaki `b` | $q_{12}$ | $a$ okundu, $b$ yazıldı, sola gidildi |
| 2 | `bbb` | ortadaki `b` | $q_2$ | $b$ okundu, $b$ yazıldı, sağa gidildi |

Sağa dönerken şerit içeriği bozulmamalıdır. Örnekte soldaki sembol $b$ olduğu için $b$ okuyup $b$ yazdık. Ama hocanın uyardığı gibi böyle yazarsak bir sorun vardır: orada $b$ değil, $a$, $c$ ya da başka bir şerit sembolü de olabilir. Bu yüzden geçiş genel yazılır: olası her şerit sembolü $x$ için $x$ okunur ve aynı $x$ geri yazılır. Asıl durum diyagramında bu geçişlerin hepsi tek tek bulunmalıdır.

Ara durumun adı $q_{12}$ olabilir, çünkü $q_1$ ile $q_2$'yi birbirine bağlar. Sonuçta:

$$\delta(q_1, a) = (q_{12}, b, L)$$

$$\delta(q_{12}, x) = (q_2, x, R) \quad \text{her şerit sembolü } x \text{ için}$$

> **Ders sonrası not:** Hocanın "her şerit sembolü" ifadesine boş sembol de dahildir. Kafa girişin ilk hücresindeyse sola gidince boş sembolü okur ve aynı biçimde geri yazıp dönmelidir. Yapı $S$ geçişi olan her ($q_1$, $a$) çifti için tekrarlanır.

Diğer hamleler için hiçbir şey değişmez. Kalma seçenekli makinede $q_1$'den $q_2$'ye "$a$ okunur, $b$ yazılır, R" geçişi varsa standart makinede de aynı geçiş bulunur. Sol hareket için de durum aynıdır.

Böylece standart Turing makinelerinin kalma seçenekli makineleri benzettiği gösterilmiş olur. Kalma seçenekli makinelerin standart makineleri benzettiği zaten biliniyordu. Dolayısıyla iki sınıf eşdeğerdir.

## 11. Yarı Sonsuz Şeritli Turing Makinesi

### 11.1 Tanım

Bu çeşit, standart Turing makinesinden daha kısıtlıdır. Şerit örneğin sağda sonsuzdur, solda sonsuz değildir. Giriş şeridin başındadır, geri kalanı boş sembollerdir. Okuma-yazma kafası başta girişin başındadır.

Bu makinede kafa sol sınırdan sola gitmek isterse en soldaki konuma geri döner. Örneğin $q_1$'de $a$ görülünce $b$ yazılıp sola gidilecek ve $q_2$'ye geçilecek olsun, kafa da en soldaki hücrede bulunsun. Normalde kafanın sola gitmesi gerekir. Ama makine orada adeta kapalıdır, kafa hemen geri döner. Yani yalnız bu en soldaki hücre için kafa yerinde kalabilir. Bu bir kalma seçeneği değildir. Geri kalan her şey standart Turing makinesiyle aynıdır.

### 11.2 Teorem ve İspatın Yapısı

**Teorem:** Yarı sonsuz şeritli Turing makineleri, standart Turing makineleriyle aynı güçtedir.

Hocanın söylediği gibi ispat her zaman iki adımlıdır. Daha kolay olan önce gelir:

- **1. adım:** Standart Turing makineleri yarı sonsuz şeritli Turing makinelerini benzetir.
- **2. adım:** Yarı sonsuz şeritli Turing makineleri standart Turing makinelerini benzetir.

### 11.3 Birinci Adım: Sol Tarafı Kesmek

Yarı sonsuz şeridi sonsuz şeritle benzetmek zor değildir. Her yarı sonsuz şeritli makine için aynı içeriğe sahip bir standart makine kurulur, ama araya özel bir sembol konur. Bu sembol # olsun:

```
yarı sonsuz şerit:     a b a c ◇ ◇ ...
standart makine:   ... ◇ # a b a c ◇ ◇ ...
```

# normalde şeritte bulunmayan özel bir semboldür ve bir tür durdurucu işlevi görür. Sonra her duruma bir öz döngü (self loop) eklenir:

$$\delta(q, \#) = (q, \#, R) \quad \text{her durum } q \text{ için}$$

Kafa # sembolünü görürse aynı sembolü geri yazar ve hemen sağa geri döner. Böylece iki iş yapılmış olur: özel sembol eklenir ve bütün durumlara öz döngü eklenir. Hocanın ifadesiyle hepsi bu kadardır. Standart makinede daha çok şerit hücresi olduğu için yapılacak tek şey sol tarafı kesmektir.

### 11.4 İkinci Adım: Çok İzli Şerit Hilesi

Bu adım biraz karışıktır. Bu sorunu çözmenin en az iki yolu vardır. Birinci yol, çok izli şerit hilesi (multiple track tape trick) kullanır.

Çok izli şerit, birden çok izi (track) olan bir şerittir. Üst üste iki şerit varmış gibi düşünülebilir, ama tek bir okuma-yazma kafası vardır. Hocanın örneğinde iki iz şöyledir:

| | | | | |
|---|---|---|---|---|
| 1. iz | $a$ | $b$ | $a$ | $b$ |
| 2. iz | $c$ | $d$ | $a$ | $c$ |

Hile şuradadır: tek kafa olduğu için bir sütun aslında tek bir semboldür. İlk sütundaki sembolün adı $ac$'dir. Yani bu aslında standart bir şerit ve standart bir Turing makinesidir. Yalnız sembollerin adı iki sembolden oluşur.

Şerit alfabesi $ac$, $bd$, $aa$, $bc$ gibi çiftlerden oluşur. Bu alfabe, özgün şerit alfabesinin bütün olası çiftlerini içerir. Özgün alfabeye $\Gamma_1$, yeni alfabeye $\Gamma_2$ dersek:

$$\Gamma_2 = \{ac,\ bd,\ aa,\ ...\}$$

Her çiftin birinci parçası 1. izden, ikinci parçası 2. izden gelir. Makine normal bir Turing makinesi gibi çalışır. Örneğin $q_1$'deyken $ac$ okunursa $cc$ yazılıp sağa ya da sola gidilir ve $q_2$'ye geçilir. Hocanın özetiyle bu, yeni bir alfabeyle zenginleştirilmiş (augmented) standart bir Turing makinesidir.

> **Ders sonrası not:** Hoca özgün alfabeyi örnek olarak $\Gamma_1 = \{a, b, c\}$ diye saydı. Ama $bd$ çifti $\Gamma_2$'de bulunduğuna göre $d$ de $\Gamma_1$'de olmalıdır.

### 11.5 Hile ile Standart Şeridi Benzetmek

Standart makinede şeritte $a\,b\,c\,d\,e$ olsun, iki yanında boş semboller bulunsun. Girişin ortasında bir yer referans noktası (reference point) olarak seçilir. Kafa $a$'dadır.

Bu içerik, iki izli yarı sonsuz şeritli bir Turing makinesinde şöyle gösterilir. Her iki izin ilk hücresinde # vardır. Üst iz referans noktasının sağındaki hücreleri sırayla, alt iz solundaki hücreleri ters sırayla tutar:

```
standart şerit:     ... ◇ a b c | d e ◇ ...
                          ^

üst iz:                 # d e ◇ ◇ ...
alt iz:                 # c b a ◇ ...
                            ^
```

Kafa $a$'nın bulunduğu sütundadır. Hocanın dediği gibi # sembolleri referans noktasına karşılık gelir.

Şimdi bir adım bakalım. Standart makinede $a$ okunur, $c$ yazılır, kafa sağa gider. Sonraki konfigürasyonda şerit `c b c d e` olur ve kafa $b$'dedir. Referans noktası değişmez. Yarı sonsuz tarafta üst iz yine `d e` kalır. Alt izde ise `c b a` yerine `c b c` olur ve kafa $b$'nin sütununa gelir:

```
standart şerit:     ... ◇ c b c | d e ◇ ...
                          ^

üst iz:                 # d e ◇ ◇ ...
alt iz:                 # c b c ◇ ...
                          ^
```

Hoca boş sembolü burada "elmas" (diamond, ◇) diye okur. Yarı sonsuz taraftaki geçiş, (◇, $a$) çiftini okur ve (◇, $c$) çiftini yazar.

> **Ders sonrası not:** Hoca standart makinedeki örnek geçişi "$a$ oku, $b$ yaz, sağa git" diye okudu, oysa şeritlerdeki değişim $a$'nın $c$ olduğunu gösteriyor. Doğrusu $a \to c, R$'dir. Hoca yarı sonsuz taraftaki geçişi de "sağa git" diye verdi. Alt iz ters sırada tutulduğu için standart makinedeki sağa hareket burada sola harekettir. Doğrusu $(\diamond, a) \to (\diamond, c), L$'dir ve kafa gerçekten $b$'nin sütununa gelir.

Hoca ispatın gerisini "oldukça çok şeyi atladım" diyerek bıraktı. Kafa sola gidince ne olacağının vb. öğrenci tarafından çıkarılması gerektiğini, işin biraz dağınık ama yapılabilir olduğunu söyledi. Sonuç olarak yarı sonsuz şeritli Turing makinesi standart Turing makinesini benzetebilir.

> **Ek bilgi (derste anlatılmadı):** Atlanan kısım şöyle tamamlanır. Makine önce girişi iki izli biçime çevirir ve ilk hücreye # koyar. Hangi izde olduğunu durumlarında tutar, yani her durum (durum, iz) çiftine çoğaltılır. Alt izdeyken standart makinenin sağa

ve sola hareketleri ters çevrilir. Kafa # sütununa gelirse iz değiştirir ve bir hücre sağa geçer.

**Ders sonrası not:** Hoca bu ispatın ortasında "yarı sonsuz şeritli makinelerin standart makinelerle aynı güçte olduğunu zaten ispatladık, bu yüzden onları kullanabiliriz" dedi. Bu döngüsel bir akıl yürütmedir. Hoca hemen fark edip düzeltti: daha önce ispatlanan kalma seçenekli makinelerdi, burada ise tam da yarı sonsuz şeritli makinelerin gücü ispatlanmaktadır. İspatın kullandığı tek şey, iki izli şeridin yeni alfabeli tek bir şerit olmasıdır.

### 11.6 İkinci Yol: Kaydırma

Hoca başka bir fikir de verir. Yarı sonsuz şeridin başına özel bir sembol konur. Şeritte $a\,b\,c$ olsun ve standart makinede kafa girişin soluna geçip oraya $b$ yazmak istesin. Standart makinede sonuç `b a b c` olur.

Yarı sonsuz şeritte makine özel sembolü görünce şeridin sonuna geldiğini anlar. O zaman yazılmak istenen sembol oraya konur, geri kalan $a\,b\,c$ de bir hücre sağa kaydırılır. Şerit, başındaki özel sembolün ardından `b a b c` olur. Yani bu işi yapmanın birden çok yolu vardır.

## 12. Çok Şeritli Turing Makinesi

Çok şeritli Turing makinesi, şimdiye kadar görülen makineye çok benzer, ama birden çok şeridi vardır: 1. şerit, 2. şerit, ..., $n$. şerit. Her şerit iki yönde sonsuzdur. Her şeridin ayrı bir okuma-yazma kafası vardır ve bu kafalar birbirinden bağımsızdır. Hepsi aynı kontrol birimine bağlıdır.

Geçişler şöyle yazılır: $q_1$'deyken 1. şeritte $a$, 2. şeritte $c$, ..., $n$. şeritte $d$ varsa, 1. şeride $b$, 2. şeride $d$, $n$. şeride yine $d$ yazılır. Sonra hareketler verilir: 1. şeritte sağa, 2. şeritte sola, $n$. şeritte sola gidilir. Son olarak $q_2$'ye geçilir.

$$\delta(q_1, (a, c, \dots, d)) = (q_2, (b, d, \dots, d), (R, L, \dots, L))$$

Bu makine çok daha güçlü görünse de standart Turing makineleriyle aynı güçtedir. Hoca bunu ispatlamaz. İspatı bir yerden okuyabileceğimizi ya da kendimizin yapabileceğini söyler.

## 13. Çok Boyutlu Turing Makinesi

Bu makinede şerit örneğin iki boyutludur. Üç boyutlu Turing makinesi de olabilir. Giriş ortadadır, geri kalan her yerde boş sembol vardır ve şerit her yönde sonsuzdur.

Kafa bir hücreyi gösterir ve bu durumda daha çok hareketi vardır: sola, sağa, yukarı ve aşağı gidebilir. Hoca bu yönleri $x$ ve $y$ eksenleriyle gösterir.

Bu makine de çok daha güçlü görünür, ama standart Turing makinesinden fazla bir güç vermez. Çok boyutlu Turing makineleri standart Turing makineleriyle aynı güçtedir. Bunu da yine kendimiz ispatlayabiliriz.

## 14. Deterministik Olmayan Turing Makinesi

**Hocanın vurgusu:** Deterministik olmayan Turing makineleri önemlidir.

### 14.1 Tanım

Deterministik olmayan Turing makinesi deterministik olmayan seçimlere izin verir. Aynı $q_1$ durumunda $a$ okunduğunda farklı şeyler yapılabilir. Örneğin $b$ yazılıp sola gidilerek $q_2$'ye ya da $c$ yazılıp sağa gidilerek $q_3$'e geçilebilir:

$$\delta(q_1, a) = \{(q_2, b, L),\ (q_3, c, R)\}$$

Makine yine kendini çoğaltır (replicate). Hoca null geçişlerin olup olmadığını da kontrol eder. Tanımda null geçişlerden söz edilmediğini söyler.

### 14.2 Hesaplama Ağacı

Makine kendini çoğalttığı için deterministik olmayan bir Turing makinesinin bir hesaplama ağacı (computation tree) vardır. Standart Turing makinesinin ise yalnız bir hesaplama listesi vardır: bir konfigürasyondan diğerine geçilir ve sonunda kabul ya da ret edilir.

Ağaçtaki dallardan herhangi birinde kabul durumuna ulaşılırsa makine kabul eder. Diğer dallar önemli değildir. Örneğin bir dal ilerlemeyip ret durumunda kalabilir, bir başka dal sonsuz döngüye girebilir. Bir kabul durumuna varılmışsa hesaplama durdurulur (halt) ve giriş kabul edilir.

### 14.3 Güç ve Hız

Deterministik olmayan Turing makineleri standart Turing makineleriyle aynı güçtedir. Yeni bir güç vermezler, yeni diller kabul edemezler. Hoca buna rağmen bunların çok daha verimli, çok daha hızlı olduğunu söyler, çünkü paralel hesaplama varmış gibidir.

> **Ders sonrası not:** Hocanın "çok daha hızlı" sözü, adım sayısı açısından bir sezgidir. Kabul eden dal kısa olabilir ve makine o dalı izlemiş sayılır. Hesaplama ağacını sırayla gezen deterministik bir benzetim ise en kötü durumda ağacın yüksekliğinde üstel sayıda konfigürasyondan geçer.

## 15. Karmaşıklık Sınıfları: P ve NP

Deterministik olmayan Turing makineleri bu yüzden hesaplama kuramının ilerleyen konularında kullanılır. Hoca bu derste oraya girmeyeceklerini söyler, ama algoritma karmaşıklık sınıflarını (complexity classes) kısaca anar:

- **P sınıfı:** Polinom (polynomial) zamanda, standart bir Turing makinesi tarafından yürütülebilir.
- **NP sınıfı:** Polinom zamanda, deterministik olmayan bir Turing makinesi tarafından yürütülebilir.

Fark, standart Turing makinesi ile deterministik olmayan Turing makinesi arasındadır. Hoca kendi kendine bu makinelerin gerçekte var olup olmadığını sorar ve şöyle bağlar: İkisi de birer hesaplama modelidir (computation model), gerçek bilgisayar değildir. Standart Turing makinesi bir hesaplama modeli, deterministik olmayan Turing makinesi başka bir hesaplama modelidir. Deterministik olmayan Turing makinesi, P ve NP karmaşıklık sınıflarının tanımında kullanıldığı için önemlidir.

> **Ders sonrası not:** Hoca "P sınıfı algoritmalar" dedi. P ve NP algoritma değil problem (dil) sınıflarıdır. Doğrusu "P sınıfındaki problemler" biçimindedir.

## Dersin Toparlaması

Ders iki eksende ilerledi. İlk eksende karşılaştırma fonksiyonu sözde kodla verildi, makineler blok diyagramlarıyla birleştirildi. Buradan Turing tezine ve algoritmanın "problemi çözen bir Turing makinesi" olarak tanımına gelindi. Böylece bir algoritmanın varlığı, bir Turing makinesinin varlığı olarak konuşulabilir hale geldi.

İkinci eksende standart modelin çeşitleri incelendi. Güç, tanınabilen dil sınıfı olarak tanımlandı. İki yönlü benzetim yöntemiyle kalma seçenekli ve yarı sonsuz şeritli makinelerin standart makinelerle aynı güçte olduğu ispatlandı. Çok şeritli, çok boyutlu ve deterministik olmayan makinelerin de aynı güçte olduğu söylendi. Bunların ispatı verilmedi. Deterministik olmayan makinenin güç değil verimlilik sorusunda öne çıktığı ve P ile NP sınıflarının tanımında kullanıldığı belirtildi. Bu, dönemin son dersidir.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.